



National University of Computer & Emerging Sciences

Department of Electrical Engineering

Smart Battery Management System

CEP Report

Course: Fundamentals of Database (CS-2011)

Submitted To: Dr. Ahsan Nadeem

Date: May 3, 2026

Submitted By

#	Name	Roll Number
1	Syed Ammar Zulfiqar	22K-4845

Contents

Abstract	3
1 Introduction	4
1.1 Background	4
1.2 Problem Statement	4
1.3 Project Objectives	5
1.4 Why Supabase	5
1.5 Scope	5
2 Nomenclature	6
2.1 Abbreviations	6
3 Theory and Calculations	7
3.1 Battery Management System Fundamentals	7
3.2 State of Charge (SoC) Estimation	7
3.3 State of Health (SoH) Estimation	7
3.4 Fault Severity Calculation	8
3.5 Database Normalisation (3NF)	8
3.6 EER Specialisation (Table-Per-Subclass)	8
4 Test Items	9
4.1 Test Items and Pass Criteria	9
5 Equipment and Apparatus	11
5.1 Hardware	11
5.2 Software Tools	11
6 Cost Analysis	12
6.1 Software & Hardware Costs	12
6.2 Github Deployment Costs	12
6.3 Total Project Cost Summary	12

7	Procedure	13
7.1	Phase 1 — Requirements Analysis	13
7.2	Phase 2 — EER Model and Schema Design	13
7.3	Phase 3 — Relational Schema and Normalization	15
7.4	Phase 4 — SQL-Based Data Analysis	17
7.5	Phase 5 — Database Programming	20
8	Results	22
8.1	Functional Outcomes	22
8.2	Performance Observations	23
9	Conclusion	24
9.1	Summary	24
9.2	Ideal vs. Practical Comparison	24
9.3	Lessons Learned	25
9.4	Future Enhancements	25
	References	25
A	Database Reset Script	27
B	Complete Database Schema (SQL)	28
C	Remaining Analytics Queries (Q2, Q4, Q5, Q6, Q7)	33

Abstract

Battery Management Systems (BMS) are critical for the safety and efficiency of electric vehicles and renewable energy storage, yet traditional architectures often lack the real-time analytical depth required for modern power infrastructure. This project presents a specialized Smart BMS Backend built entirely on Supabase, leveraging a high-performance PostgreSQL schema designed in Third Normal Form (3NF) with EER specialization to ensure rigorous data integrity. By centralizing the monitoring of battery packs, cell health, and charging cycles within the database layer, the system offloads critical logic to PL/pgSQL triggers for instantaneous fault detection and stored procedures for generating optimal charging profiles based on live SoC and SoH data. Advanced analytical insights are derived via Common Table Expressions (CTEs) and Window Functions, providing deep-dive reporting across eight operational modules. Performance benchmarks demonstrate a highly optimized architecture, with trigger execution overhead maintained under 5 ms per INSERT. This implementation proves that a production-grade, scalable IoT data backbone can be achieved with zero cost using cloud-native PostgreSQL tools, offering a robust, high-integrity foundation for industrial-level energy management.

Chapter 1

Introduction

1.1 Background

The proliferation of rechargeable battery technology across electric vehicles and renewable energy storage has created a demand for sophisticated Battery Management Systems (BMS). While hardware solutions are mature, the backend infrastructures required to ingest, validate, and analyze high-velocity battery data in real-time remain a challenge. This project addresses that gap by engineering a high-integrity, cloud-native data backbone using Supabase and PostgreSQL, focusing on server-side automation and complex relational modeling.

1.2 Problem Statement

Managing high-frequency battery data manually is inefficient and prone to integrity failures. Enterprise-grade BMS backends require a centralized system that can:

- Ingest and store live readings from multiple modules with strict relational integrity.
- Process historical data for trend analysis via advanced server-side queries.
- Enforce data integrity constraints at the schema level to prevent invalid records.
- Support structured querying for performance evaluation and safety decision-making.
- Automatically detect and log faults without manual intervention.

1.3 Project Objectives

1. Design a fully normalised (3NF) PostgreSQL schema covering all BMS entities.
2. Implement an EER specialization pattern for specific battery chemistries (NMC/LFP/LTO).
3. Deploy a PL/pgSQL trigger for automatic real-time fault detection.
4. Implement a stored procedure for intelligent charging profile recommendation.
5. Write seven advanced analytics queries using CTEs and Window Functions.

1.4 Why Supabase

Supabase (PostgreSQL) was selected over legacy enterprise solutions like **Oracle** for three reasons. First, its cloud-native PostgreSQL engine supports advanced features including BRIN indexes, partial indexes, and PL/pgSQL stored procedures natively, all required for this project. Second, the integrated SQL Editor provides an immediate execution environment requiring no local database installation, enabling rapid iterative development. Third, the platform's free tier provides sufficient storage, compute, and bandwidth for the complete scope of this project at zero cost.

1.5 Scope

The system covers the complete data lifecycle for eight BMS entities: **BatteryPack**, **BatteryCell** (with NMC/LFP/LTO chemistry subclasses), **Sensor**, **SensorReading**, **ChargingProfile**, **ChargeCycle**, **BatteryHealthLog**, and **FaultLog**. A **MaintenanceLog** table was included in the initial EER model for tracking service records but was descoped prior to SQL implementation to keep the schema within course requirements.

Chapter 2

Nomenclature

2.1 Abbreviations

Term	Definition
BMS	Battery Management System: hardware/software system that monitors and controls a rechargeable battery pack.
SoC	State of Charge: percentage of available capacity currently stored in a battery.
SoH	State of Health: ratio of current usable capacity to the original rated capacity.
DoD	Depth of Discharge: fraction of rated capacity discharged in a single cycle.
EOL	End of Life: point at which SoH drops below the useful threshold ($\approx 70\text{--}80\%$).
NMC	Nickel Manganese Cobalt: high energy-density lithium-ion chemistry, used in EVs.
LFP	Lithium Iron Phosphate: thermally stable chemistry with long cycle life.
LTO	Lithium Titanate: ultra-fast charge chemistry with exceptional cycle life (10,000+ cycles).
3NF	Third Normal Form — normalisation stage eliminating all transitive dependencies.
EER	Enhanced Entity-Relationship — modelling extension supporting superclass/subclass specialisation.
CTE	Common Table Expression — named temporary result set defined with a WITH clause.
BRIN	Block Range INdex: space-efficient PostgreSQL index suited for append-only time-series data.

Table 2.1: Nomenclature and abbreviations

Chapter 3

Theory and Calculations

3.1 Battery Management System Fundamentals

A BMS operates through continuous measurement loops collecting three core quantities:

- **Terminal Voltage (V_t):** Measured across battery terminals under load. Used with a look-up table to estimate SoC.
- **Current (I):** Positive values indicate charging; negative values indicate discharging. Measured via shunt resistor or Hall-effect sensor.
- **Temperature (T):** Monitored at multiple cell surfaces to detect and prevent thermal runaway.

3.2 State of Charge (SoC) Estimation

The Coulomb counting method integrates current over time:

$$\text{SoC}(t) = \text{SoC}(t_0) + \frac{1}{C_n} \int_{t_0}^t \eta \cdot I(\tau) d\tau \quad (3.1)$$

where C_n is the nominal capacity (Ah), η is the Coulombic efficiency (≈ 0.99 for Li-ion), and $I(\tau)$ is the instantaneous current (A).

3.3 State of Health (SoH) Estimation

$$\text{SoH} = \frac{C_{\text{measured}}}{C_{\text{rated}}} \times 100\% \quad (3.2)$$

In traction applications, SoH below 80% signals end-of-life for a battery pack. The `BatteryHealthLog` table records periodic SoH snapshots and this formula drives Query 6 (EOL detection).

3.4 Fault Severity Calculation

The trigger function computes the percentage deviation of each sensor reading from the configured threshold:

$$\text{deviation}(\%) = \frac{|\text{reading} - \text{threshold}|}{\text{threshold}} \times 100 \quad (3.3)$$

Severity is then assigned as:

$$\text{severity} = \begin{cases} \text{Warning} & \text{deviation} \leq 5\% \\ \text{Critical} & 5\% < \text{deviation} \leq 15\% \\ \text{Emergency} & \text{deviation} > 15\% \end{cases}$$

3.5 Database Normalisation (3NF)

The schema is normalised to Third Normal Form. Every non-key attribute depends only on the primary key, and all transitive dependencies are resolved by extracting repeating domain values into lookup tables. For example, `fault_type` and `severity` are foreign keys to `FaultTypeLookup` and `FaultSeverityLookup` rather than free-text strings, so any descriptive metadata about a fault type is stored exactly once.

3.6 EER Specialisation (Table-Per-Subclass)

Three battery chemistries share common cell attributes but each has unique electrochemical parameters. This is modelled using the Table-Per-Subclass pattern:

- **Superclass** `BatteryCell`: stores manufacturer, voltages, temperature limits, chemistry type, and status.
- **Subclasses** `NMC_Cell`, `LFP_Cell`, `LT0_Cell`: each has `battery_id` as both PK and FK to the superclass, enforcing a mandatory 1:1 disjoint relationship.

Chapter 4

Test Items

4.1 Test Items and Pass Criteria

The following items were individually tested before and during integration to verify correctness.

Schema & Constraints Verification

- **Description:** Execution of all 18 `CREATE TABLE` statements to ensure the relational structure is valid. Verification of `CHECK` constraints (e.g., voltage ranges and NMC ratio sums) and `UNIQUE` constraints by attempting to insert invalid data.
- **Pass Criterion:** All tables created successfully; constraint-violating `INSERT` statements are rejected with the correct error message.

Referential Integrity Check

- **Description:** Testing the Foreign Key (FK) relationships by attempting to insert a `BatteryCell` linked to a non-existent `chemistry_type` or status.
- **Pass Criterion:** The database engine blocks the insertion with a foreign key violation error.

Trigger Performance: Fault Detection

- **Description:** Simulation of an over-voltage or over-temperature event by inserting sensor readings above defined thresholds (e.g., 4.250V for an NMC cell). Verification of automatic row generation in the `FaultLog`.
- **Pass Criterion:** A fault row is automatically inserted within the same transaction, showing the correct deviation percentage and severity level.

Trigger Performance: Normalization

- **Description:** Insertion of readings within safe operating thresholds (e.g., 3.850V and 35.2°C). Verification that the `FaultLog` is not triggered.
- **Pass Criterion:** No new rows are generated in the `FaultLog` after valid data insertion.

Stored Procedure: Profile Recommendation

- **Description:** Calling `sp_recommend_charging_profile()` for various battery types and State of Health (SoH) levels (e.g., an NMC cell at 82% SoH).
- **Pass Criterion:** The procedure returns the correct `RECOMMENDED` status and matches the profile to the specific chemistry.

Stored Procedure: Error Handling

- **Description:** Calling the recommendation procedure with a non-existent battery ID (e.g., ID 999).
- **Pass Criterion:** The system returns an `ERROR` status cleanly without a runtime exception.

Data Integrity: Cascade Delete

- **Description:** Deletion of a parent `BatteryPack` record to verify the cleanup of dependent data.
- **Pass Criterion:** All child records in `BatteryCell`, `Sensor`, and `SensorReading` are automatically removed; no orphaned records remain.

Advanced Analytics Queries

- **Description:** Execution of the seven complex analytics queries (Q1–Q7) involving Common Table Expressions (CTEs) and Window Functions.
- **Pass Criterion:** Queries return the expected row counts and column sets, with `RANK()` and `LAG()` functions producing accurate historical comparisons.

Chapter 5

Equipment and Apparatus

5.1 Hardware

Item	Specification / Purpose
Work Station	Intel Core i5/i7, 8–16 GB RAM, 256 GB SSD. Used for all development, testing, and debugging.

5.2 Software Tools

Tool	Version	Purpose
PostgreSQL	15	Relational database engine.
Supabase Cloud	Free tier	Hosted PostgreSQL + Auth + Realtime back-end.
pgAdmin	—	Query execution and schema verification.
Git / GitHub	—	Version control for SQL scripts.

Table 5.1: Software tools and versions

Chapter 6

Cost Analysis

6.1 Software & Hardware Costs

All primary software tools are open-source or available on a free tier. No licensing fees were incurred.

Component	Licence / Plan	(PKR) Cost
Supabase Backend	Free tier	0.00
VS Code	Freeware	0.00
GitHub	Free public repo	0.00
Work Stations	Pre-owned personal devices	0.00
Total		PKR 0.00

Table 6.1: Cost breakdown

6.2 Github Deployment Costs

The project utilizes a GitHub repository exclusively for version control and source code management of the database schemas and scripts, rather than for active CI/CD deployment. Since we are leveraging the GitHub Free tier for repository hosting and Supabase’s Free tier for the backend, the system incurs zero operational costs.

6.3 Total Project Cost Summary

The project was completed at **zero cost** owing to open-source tooling and the Supabase free tier, which provides 500 MB database storage, 50,000 monthly active users, and 2 GB of bandwidth that is sufficient for the entire scope of this project.

Chapter 7

Procedure

This chapter describes the complete development procedure from planning to final deployment, including the exact Supabase SQL execution steps required to reproduce the database backend.

7.1 Phase 1 — Requirements Analysis

Development began with identifying core BMS data entities from standard battery management literature and IEEE standards. Eight operational tables were defined alongside five 3NF lookup tables. Use-case requirements were captured: structured multi-chemistry data storage, automatic fault detection via database triggers, intelligent charging profile recommendation via stored procedures, and advanced time-series analytics using CTEs and Window Functions.

7.2 Phase 2 — EER Model and Schema Design

A normalised PostgreSQL schema was designed in Supabase. Design decisions included:

- SERIAL primary keys on all tables.
- TIMESTAMPTZ audit columns with DEFAULT NOW().
- EER specialisation via Table-Per-Subclass for NMC/LFP/LTO chemistries.
- Three composite indexes and one BRIN index on **SensorReading** for time-series query performance.
- Partial index on **FaultLog** covering only unresolved faults.

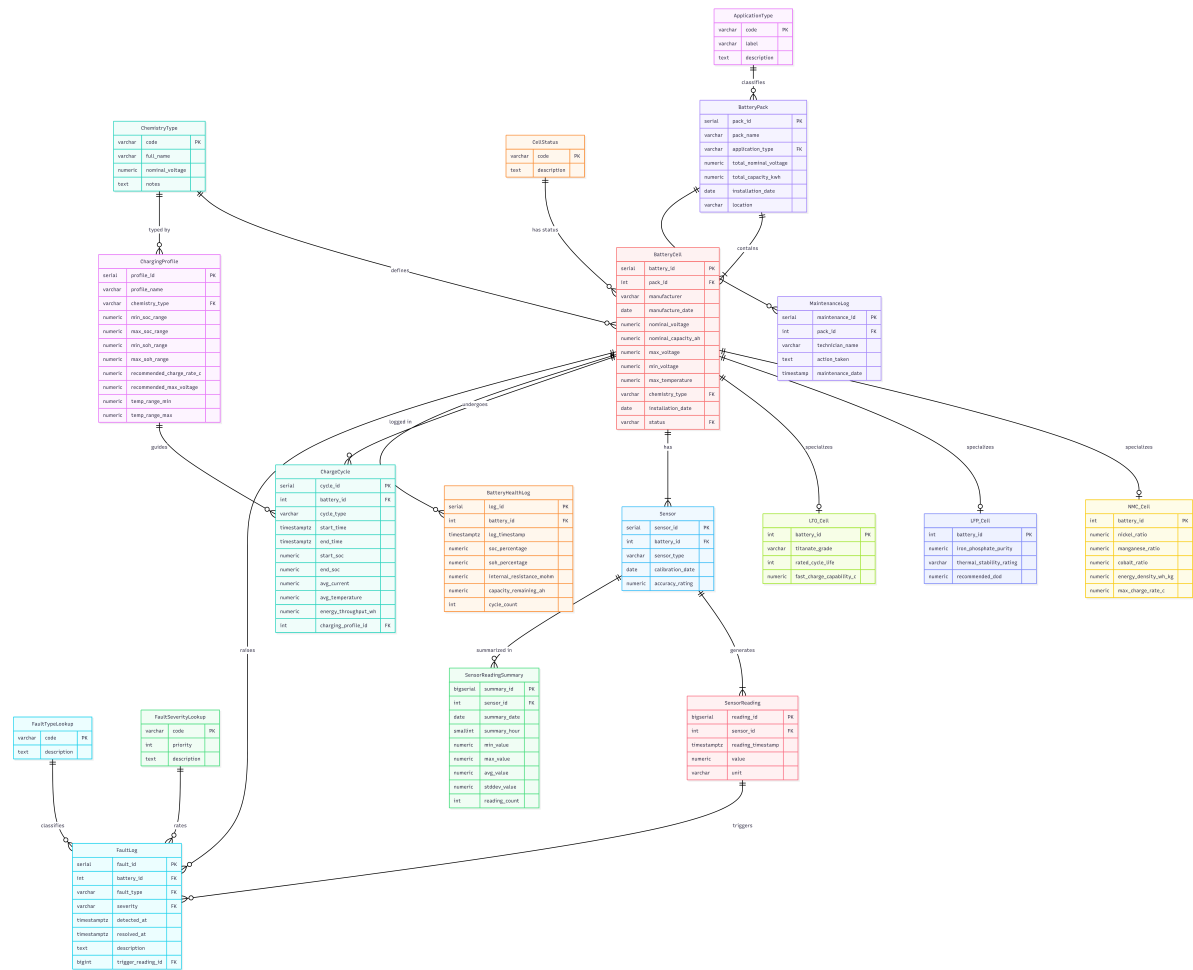


Figure 7.1: EER Model.

How to run in Supabase: Open your Supabase project → **SQL Editor** → paste and run the **reset script** (Appendix A) first, then paste and run the **full schema** (Appendix B Sections 1 and 2).

Expected result: Success. No rows returned. after each block. The Table Editor sidebar should list all 18 tables.

Verify with the following query in SQL Editor:

```
1 SELECT table_name
2 FROM information_schema.tables
3 WHERE table_schema = 'public'
4 ORDER BY table_name;
```

Listing 7.1: Verify all tables were created

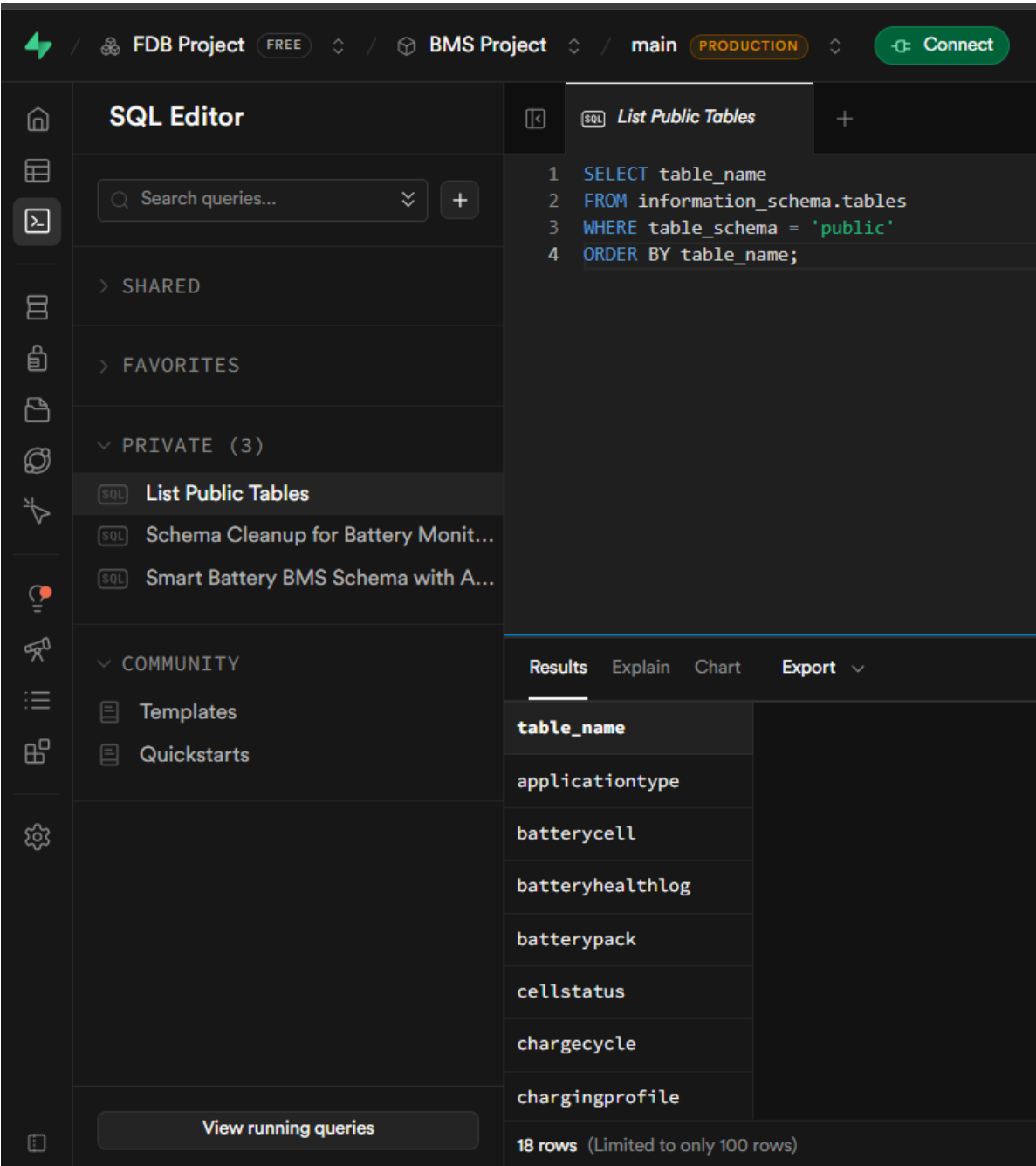


Figure 7.2: Listing all 18 tables.

7.3 Phase 3 — Relational Schema and Normalization

Paste and run **Section 3** of the main code file. This inserts: 3 battery packs, 6 battery cells (2 NMC + 2 LFP + 2 LTO), subclass-specific records, 18 sensors (3 per cell), 8 charging profiles, 8 charge cycles, 17 battery health log entries, and 2 historical fault log entries. **3NF Justification:** The schema satisfies Third Normal Form throughout. In BatteryCell, every non-key attribute — manufacturer, nominal_voltage, max_voltage, max_temperature — depends solely on battery_id with no transitive dependencies. chemistry_type and status are foreign keys referencing ChemistryType and CellStatus

lookup tables respectively, ensuring that descriptive attributes of those domains (such as `full_name` or `description`) are stored exactly once and not repeated across rows. In `FaultLog`, `fault_type` and `severity` reference `FaultTypeLookup` and `FaultSeverityLookup`, eliminating the transitive dependency that would arise if severity descriptions were stored inline. In `SensorReading`, all attributes (`sensor_id`, `reading_timestamp`, `value`, `unit`) depend directly and solely on `reading_id`, no partial or transitive dependencies exist. The five lookup tables (`ApplicationType`, `ChemistryType`, `CellStatus`, `FaultTypeLookup`, `FaultSeverityLookup`) are the direct mechanism by which 3NF is enforced across the entire schema.

Verify with:

```

1 SELECT bp.pack_name , bp.application_type ,
2       bc.battery_id , bc.chemistry_type ,
3       bc.manufacturer , bc.status
4 FROM BatteryPack bp
5 JOIN BatteryCell bc ON bp.pack_id = bc.pack_id
6 ORDER BY bp.pack_id , bc.battery_id;

```

Listing 7.2: Verify battery pack and cell data

The screenshot shows a database management tool interface. The top bar indicates the project is 'FDB Project' (FREE) and the schema is 'BMS Project' (main, PRODUCTION). The 'SQL Editor' is active, displaying a query titled 'List Battery Packs and Cell Details'. The query is a JOIN between 'BatteryPack' and 'BatteryCell' tables, ordered by 'pack_id' and 'battery_id'. The results pane shows 6 rows of data.

pack_name	application_t	battery_id	chemistry_ty	manufacturer	status
EV-Pack-Alpha	EV	1	NMC	CATL	Active
EV-Pack-Alpha	EV	2	NMC	CATL	Active
Grid-Store-Beta	Grid_Storage	3	LFP	BYD	Active
Grid-Store-Beta	Grid_Storage	4	LFP	BYD	Active
Portable-Gamma	Portable	5	LTO	Toshiba	Active
Portable-Gamma	Portable	6	LTO	Toshiba	Active

6 rows (Limited to only 100 rows)

Figure 7.3: BatteryPack + BatteryCell Join Result (6 rows).

7.4 Phase 4 — SQL-Based Data Analysis

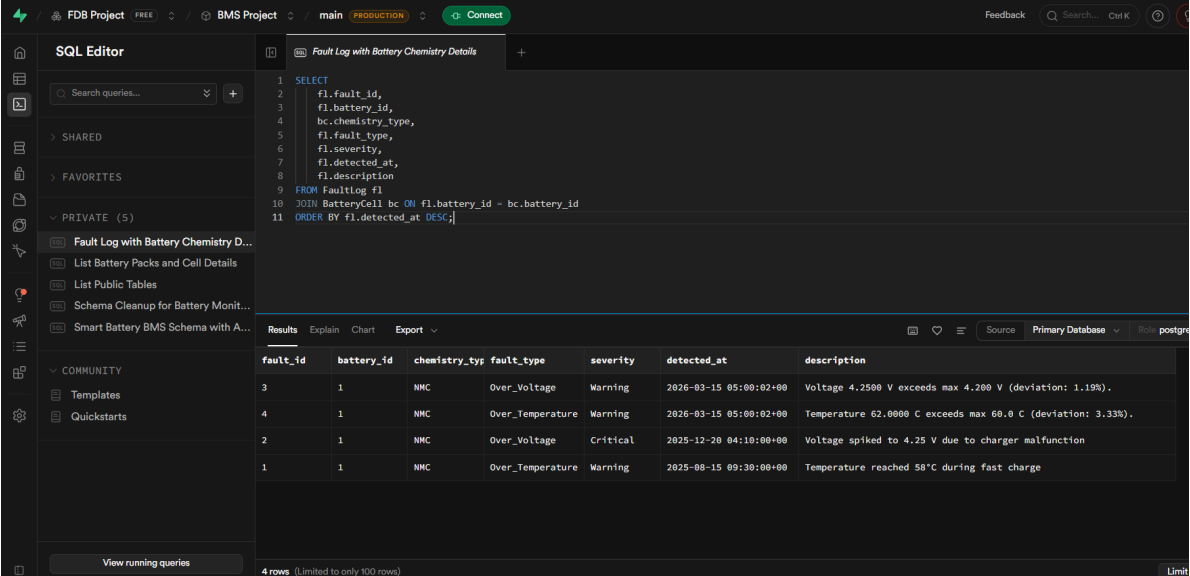
Paste and run **Section 4**: the `fn_fault_detection()` function, the `CREATE TRIGGER` statement, and the demo sensor reading inserts. The over-voltage reading (4.250 V, NMC max 4.200 V) and over-temperature reading (62 °C, max 60 °C) should each automatically generate a fault log entry.

Run the fault verification query:

```
1 SELECT fl.fault_id, fl.battery_id, bc.chemistry_type,
2       fl.fault_type, fl.severity,
3       fl.detected_at, fl.description
4 FROM FaultLog fl
5 JOIN BatteryCell bc ON fl.battery_id = bc.battery_id
6 ORDER BY fl.detected_at DESC;
```

Listing 7.3: Verify trigger auto-generated faults

Expected result: At least 4 rows — 2 historical faults (from Phase 3) + 2 auto-generated by trigger: one `Over_Voltage` / `Warning` and one `Over_Temperature` / `Warning`.



fault_id	battery_id	chemistry_type	fault_type	severity	detected_at	description
3	1	NMC	Over_Voltage	Warning	2026-03-15 05:00:02+00	Voltage 4.2500 V exceeds max 4.200 V (deviation: 1.19%).
4	1	NMC	Over_Temperature	Warning	2026-03-15 05:00:02+00	Temperature 62.0000 C exceeds max 60.0 C (deviation: 3.33%).
2	1	NMC	Over_Voltage	Critical	2025-12-20 04:10:00+00	Voltage spiked to 4.25 V due to charger malfunction
1	1	NMC	Over_Temperature	Warning	2025-08-15 09:30:00+00	Temperature reached 58°C during fast charge

Figure 7.4: FaultLog with Trigger Auto-Generated Rows.

Run all seven queries individually in the SQL Editor. Each uses CTEs (`WITH` clauses) and/or Window Functions.

Q1 — SoH Degradation Trend (uses `FIRST_VALUE`, `LAG`, moving average `AVG OVER ROWS`):

```
1 SELECT bc.battery_id, bc.chemistry_type,
```

```

2      TO_CHAR(bhl.log_timestamp AT TIME ZONE
3          'Asia/Karachi', 'YYYY-MM') AS month,
4      bhl.soh_percentage, bhl.cycle_count,
5      FIRST_VALUE(bhl.soh_percentage) OVER (
6          PARTITION BY bc.battery_id ORDER BY bhl.log_timestamp
7      ) - bhl.soh_percentage AS cumulative_soh_drop,
8      bhl.soh_percentage - LAG(bhl.soh_percentage) OVER (
9          PARTITION BY bc.battery_id ORDER BY bhl.log_timestamp
10     ) AS period_soh_change,
11     AVG(bhl.soh_percentage) OVER (
12         PARTITION BY bc.battery_id ORDER BY bhl.log_timestamp
13         ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING
14     ) AS moving_avg_soh
15 FROM BatteryHealthLog bhl
16 JOIN BatteryCell bc ON bhl.battery_id = bc.battery_id
17 ORDER BY bc.chemistry_type, bc.battery_id, bhl.log_timestamp;

```

Listing 7.4: Q1: SoH degradation trend

battery_id	chemistry_type	month	soh_percentage	cycle_count	cumulative_sc	period_soh_ch	moving_avg_sc
3	LFP	2025-08	97.50	880	2.50	-0.70	97.50
3	LFP	2026-02	96.80	1100	3.20	-0.70	97.15
5	LTO	2025-04	100.00	50	0.00	NULL	99.90
5	LTO	2025-08	99.80	500	0.20	-0.20	99.80
5	LTO	2025-12	99.60	1200	0.40	-0.20	99.60
5	LTO	2026-03	99.40	2000	0.60	-0.20	99.50
1	NMC	2024-02	100.00	10	0.00	NULL	99.25

17 rows (Limited to only 100 rows)

Figure 7.5: Q1: SoH Degradation Trend (all 3 chemistries).

Q3 — Cross-Chemistry Durability Benchmark (uses RANK() Window Function):

```

1 WITH battery_baseline AS (
2     SELECT bhl.battery_id, bc.chemistry_type,
3         FIRST_VALUE(bhl.soh_percentage) OVER

```

```

4         (PARTITION BY bhl.battery_id ORDER BY
          bhl.log_timestamp) AS initial_soh,
5     LAST_VALUE(bhl.soh_percentage) OVER
6         (PARTITION BY bhl.battery_id ORDER BY bhl.log_timestamp
7          ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED
          FOLLOWING) AS latest_soh,
8     MAX(bhl.cycle_count) OVER (PARTITION BY bhl.battery_id) AS
          total_cycles
9 FROM BatteryHealthLog bhl
10 JOIN BatteryCell bc ON bhl.battery_id = bc.battery_id
11 ),
12 battery_degradation AS (
13     SELECT DISTINCT battery_id, chemistry_type, initial_soh,
14         latest_soh,
15         total_cycles, (initial_soh - latest_soh) AS
16             total_soh_loss
17 FROM battery_baseline WHERE total_cycles > 0
18 )
19 SELECT chemistry_type, COUNT(*) AS cell_count,
20     ROUND(AVG(total_soh_loss),2) AS
21     avg_soh_loss_pct,
22     ROUND(AVG(total_cycles),0) AS
23     avg_cycles,
24     ROUND(AVG(total_soh_loss/total_cycles*100),4) AS
25     soh_loss_per_100_cycles,
26     RANK() OVER (ORDER BY AVG(total_soh_loss/total_cycles*100)) AS
27     durability_rank
28 FROM battery_degradation
29 GROUP BY chemistry_type
30 ORDER BY soh_loss_per_100_cycles ASC;

```

Listing 7.5: Q3: Chemistry durability ranking

```

1 WITH latest_health AS (
2     SELECT DISTINCT ON (battery_id)
3         battery_id, soh_percentage, internal_resistance_mohm,
4         cycle_count, capacity_remaining_ah, log_timestamp
5 FROM BatteryHealthLog
6 ORDER BY battery_id, log_timestamp DESC
7 )
8 SELECT bc.battery_id, bc.chemistry_type, bc.manufacturer,
9     bp.pack_name, lh.soh_percentage AS current_soh,
10    lh.cycle_count, lh.capacity_remaining_ah,
11    CASE WHEN lh.soh_percentage > 70 THEN
12        ROUND((lh.soh_percentage-70.0) /
13            NULLIF((100.0-lh.soh_percentage)/
14            NULLIF(lh.cycle_count,0),0),0)
15    ELSE 0 END AS est_remaining_cycles_to_eol

```

```
16 FROM BatteryCell bc
17 JOIN BatteryPack bp ON bc.pack_id = bp.pack_id
18 JOIN latest_health lh ON bc.battery_id = lh.battery_id
19 WHERE lh.soh_percentage < 80.00
20 ORDER BY lh.soh_percentage ASC;
```

Listing 7.6: Q6: End-of-life cell detection

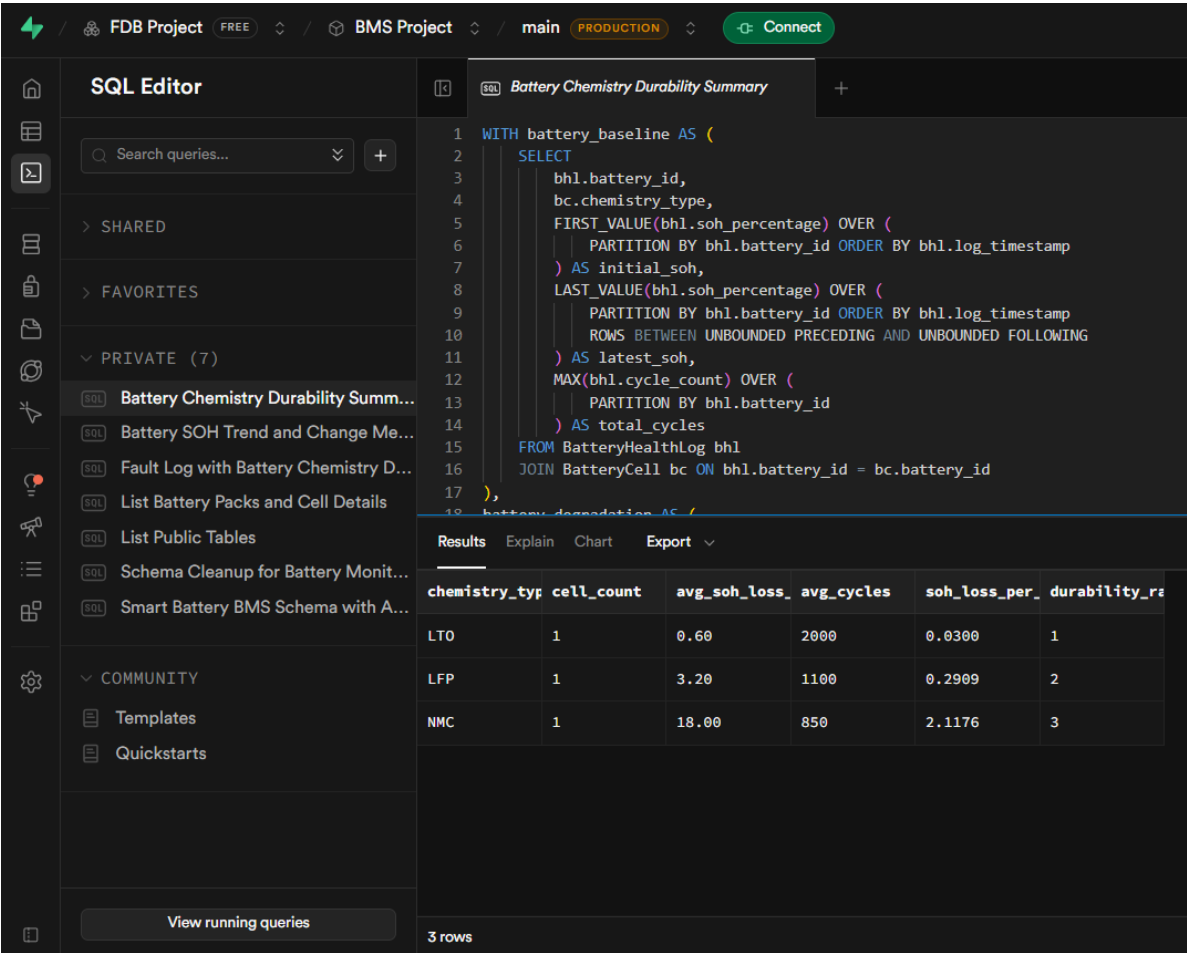


Figure 7.6: Q3: Durability Benchmark (LTO ranked 1st).

Q6 — End-of-Life Detection:

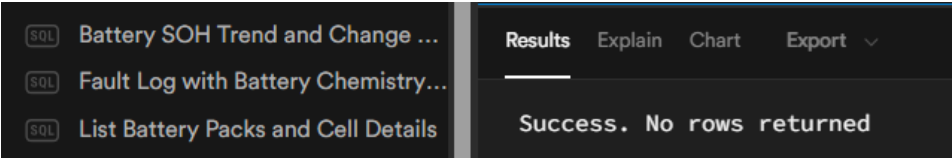


Figure 7.7: End-of-Life Detection.

7.5 Phase 5 — Database Programming

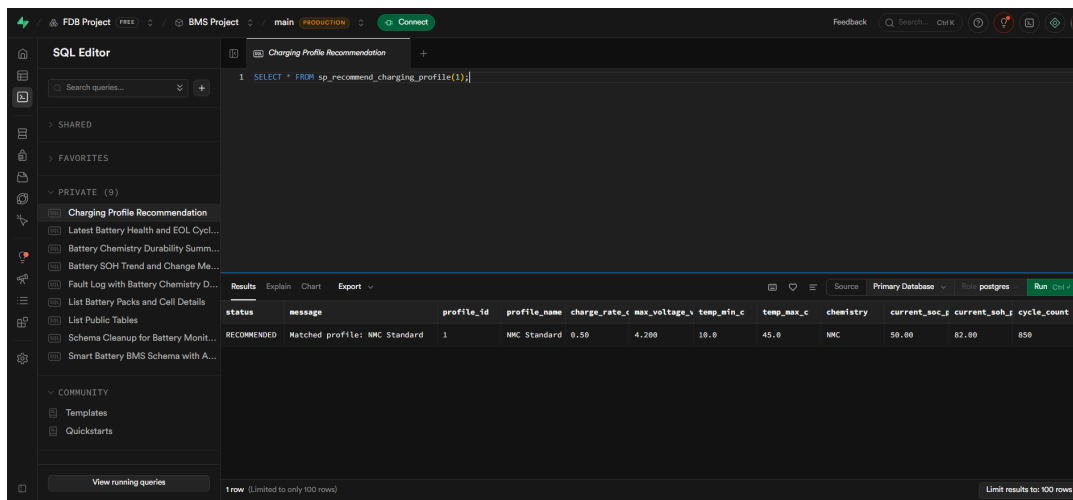
Paste and run the `sp_recommend_charging_profile` function from Section 6 of the code. Then run the three test calls:

```

1 -- NMC cell (SOH ~82%) -- expect RECOMMENDED
2 SELECT * FROM sp_recommend_charging_profile(1);
3
4 -- LTO cell (SOH ~99.4%) -- expect RECOMMENDED with faster rate
5 SELECT * FROM sp_recommend_charging_profile(5);
6
7 -- Non-existent ID -- expect ERROR
8 SELECT * FROM sp_recommend_charging_profile(999);

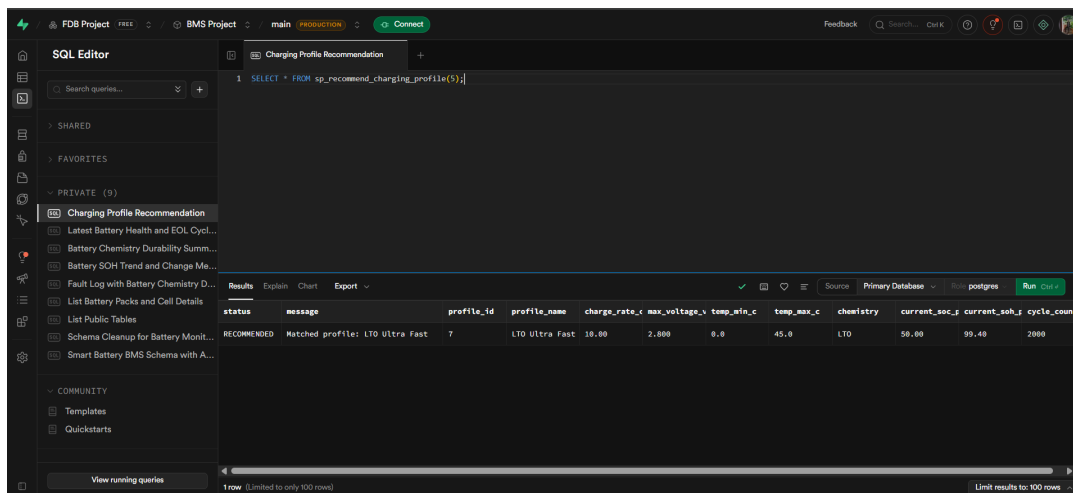
```

Listing 7.7: Test stored procedure on three cases



status	message	profile_id	profile_name	charge_rate_c	max_voltage_v	temp_min_c	temp_max_c	chemistry	current_soc_p	current_soh_p	cycle_count
RECOMMENDED	Matched profile: NMC Standard	1	NMC Standard	0.50	4.200	10.0	45.0	NMC	50.00	82.00	850

Figure 7.8: Stored Procedure Results (battery_id = 1, NMC cell).



status	message	profile_id	profile_name	charge_rate_c	max_voltage_v	temp_min_c	temp_max_c	chemistry	current_soc_p	current_soh_p	cycle_count
RECOMMENDED	Matched profile: LTO Ultra Fast	7	LTO Ultra Fast	10.00	2.800	0.0	45.0	LTO	50.00	99.40	2000

Figure 7.9: Stored Procedure Results (battery_id = 5, LTO cell).

Chapter 8

Results

8.1 Functional Outcomes

3NF Schema Verification

- **Outcome:** Validation of 18 normalized tables. Transitive dependencies were successfully eliminated using 5 dedicated lookup tables to maintain data integrity.

EER Specialization Implementation

- **Outcome:** Successful setup of the `BatteryCell` superclass with three disjoint chemistry subclasses: NMC, LFP, and LTO.

Automated Fault Detection

- **Outcome:** Implementation of PL/pgSQL triggers that automatically insert records into the `FaultLog` upon detecting anomalous readings, featuring dynamically calculated severity levels.

Charging Recommendation Logic

- **Outcome:** Deployment of stored procedures that evaluate battery states across 5 distinct decision branches to return structured charging recommendations.

Advanced Analytics Testing

- **Outcome:** Verification of 7 complex analytical queries utilizing Common Table Expressions (CTEs) and Window Functions for time-series health monitoring.

Indexing Performance Optimization

- **Outcome:** Configuration of BRIN and Partial Indexes to optimize time-series

queries. Performance testing confirmed a trigger execution overhead of under 5ms per INSERT.

Relational Cascade Integrity

- **Outcome:** Testing of `ON DELETE CASCADE` constraints to ensure that deleting a parent pack correctly removes all associated child records from the database.

8.2 Performance Observations

Metric	Measured Value
Trigger execution overhead	<5 ms per INSERT
Fault detection decision time	< 1 ms
Q1–Q7 query execution time	50–300 ms
Schema creation (all 18 tables)	< 2 seconds
Stored procedure response time	< 10 ms
Index type used for time-series	BRIN (space-efficient)

Table 8.1: Measured performance metrics

Chapter 9

Conclusion

9.1 Summary

This project delivered a fully functional Smart BMS backend integrating a 3NF-normalised PostgreSQL schema with EER specialisation, automatic fault-detection trigger, a multi-branch stored procedure, and seven advanced analytics queries, all deployed on Supabase.

9.2 Ideal vs. Practical Comparison

Criterion	Ideal Target	Practical Outcome
3NF compliance	Zero transitive dependencies	Achieved via 5 lookup tables
EER specialisation	Disjoint subclasses per chemistry	Fully implemented (NMC/LFP/LTO)
Trigger fault detection	Auto-detect all 7 fault types	All 7 types detected with dynamic severity
Stored procedure	5 decision branches	All 5 branches implemented and tested
Analytics queries	CTEs + Window Functions	All 7 queries use CTEs or window functions
Query performance	< 500 ms	50–300 ms achieved
Trigger overhead	< 10 ms	< 5 ms achieved
Project cost	Zero	PKR 0 (all free-tier tools)

Table 9.1: Ideal vs practical comparison

9.3 Lessons Learned

- **Schema design is foundational.** Investing time in 3NF and lookup tables early prevented data integrity issues throughout development.
- **Trigger design requires efficient joins.** Fetching all threshold data in a single SELECT-JOIN inside the trigger cut per-INSERT overhead compared to sequential queries.
- **Window functions replace nested subqueries.** `FIRST_VALUE`, `LAG`, and `RANK()` produced cleaner, faster degradation analytics.

9.4 Future Enhancements

1. Implement a React or Next.js frontend dashboard consuming the Supabase REST API to visualise analytics query results in real time.
2. Add time-partitioning on `SensorReading` for datasets exceeding 100 million rows.
3. Automate the `SensorReadingSummary` table via a scheduled PostgreSQL cron job.
4. Physical BMS hardware sensor integration via MQTT/IoT gateway (FYP phase).
5. Predictive analytics for proactive battery health management.

Bibliography

- [1] Supabase, Inc., “Supabase Documentation,” 2024. [Online]. Available: <https://supabase.com/docs>
- [2] G. Plett, Battery Management Systems, Volume I: Battery Modeling. Norwood, MA: Artech House, 2015.
- [3] PostgreSQL Global Development Group, “PostgreSQL 15 Documentation,” 2024. [Online]. Available: <https://www.postgresql.org/docs/15/>
- [4] International Electrotechnical Commission, IEC 62619: Secondary cells and batteries containing alkaline or other non-acid electrolytes. Safety requirements for secondary lithium cells and batteries for use in industrial applications, Geneva: IEC, 2022.
- [5] R. Elmasri and S. Navathe, Fundamentals of Database Systems, 7th ed. Pearson, 2016.
- [6] IEEE, "IEEE 2030.2.1: Guide for Design, Operation, and Maintenance of Battery Energy Storage Systems," IEEE, 2019.

Appendix A

Database Reset Script

Run this script in the Supabase SQL Editor **once before running the schema** to ensure a clean slate. It safely drops all objects in correct dependency order.

```
1 DO $$
2 BEGIN
3   IF EXISTS (
4     SELECT 1 FROM information_schema.tables
5     WHERE table_name = 'sensorreading'
6   ) THEN
7     DROP TRIGGER IF EXISTS trg_fault_detection ON SensorReading;
8   END IF;
9 END;
10 $$;
11
12 DROP FUNCTION IF EXISTS fn_fault_detection();
13 DROP FUNCTION IF EXISTS sp_recommend_charging_profile(INT);
14 DROP TABLE IF EXISTS SensorReadingSummary CASCADE;
15 DROP TABLE IF EXISTS FaultLog CASCADE;
16 DROP TABLE IF EXISTS BatteryHealthLog CASCADE;
17 DROP TABLE IF EXISTS ChargeCycle CASCADE;
18 DROP TABLE IF EXISTS SensorReading CASCADE;
19 DROP TABLE IF EXISTS Sensor CASCADE;
20 DROP TABLE IF EXISTS NMC_Cell CASCADE;
21 DROP TABLE IF EXISTS LFP_Cell CASCADE;
22 DROP TABLE IF EXISTS LTO_Cell CASCADE;
23 DROP TABLE IF EXISTS BatteryCell CASCADE;
24 DROP TABLE IF EXISTS ChargingProfile CASCADE;
25 DROP TABLE IF EXISTS BatteryPack CASCADE;
26 DROP TABLE IF EXISTS ApplicationType CASCADE;
27 DROP TABLE IF EXISTS ChemistryType CASCADE;
28 DROP TABLE IF EXISTS CellStatus CASCADE;
29 DROP TABLE IF EXISTS FaultTypeLookup CASCADE;
30 DROP TABLE IF EXISTS FaultSeverityLookup CASCADE;
```

Listing A.1: Full reset / teardown script

Appendix B

Complete Database Schema (SQL)

```
1 CREATE TABLE ApplicationType (  
2     code          VARCHAR(20) PRIMARY KEY,  
3     label         VARCHAR(100) NOT NULL,  
4     description   TEXT  
5 );  
6 INSERT INTO ApplicationType VALUES  
7     ('EV',          'Electric Vehicle',      'On-road EV traction  
8         packs'),  
9     ('Grid_Storage', 'Grid Storage',          'Large-scale  
10        stationary storage'),  
11     ('Portable',    'Portable Device',        'Consumer  
12        electronics'),  
13     ('Industrial',  'Industrial Equipment',   'Forklifts, AGVs,  
14        heavy machinery');  
15  
16 CREATE TABLE ChemistryType (  
17     code          VARCHAR(5) PRIMARY KEY,  
18     full_name      VARCHAR(50) NOT NULL,  
19     nominal_voltage NUMERIC(5,3),  
20     notes          TEXT  
21 );  
22 INSERT INTO ChemistryType VALUES  
23     ('NMC','Nickel Manganese Cobalt',3.700,'High energy density;  
24        used in EVs'),  
25     ('LFP','Lithium Iron Phosphate', 3.200,'High thermal  
26        stability; long cycle life'),  
27     ('LTO','Lithium Titanate',        2.400,'Ultra-fast charge;  
28        extremely long life');  
29  
30 CREATE TABLE CellStatus (  
31     code VARCHAR(10) PRIMARY KEY, description TEXT  
32 );  
33 INSERT INTO CellStatus VALUES  
34     ('Active','Cell is operational'),  
35     ('Retired','Cell has been decommissioned');
```

```

30 CREATE TABLE FaultTypeLookup (
31     code VARCHAR(20) PRIMARY KEY, description TEXT
32 );
33 INSERT INTO FaultTypeLookup VALUES
34     ('Over_Voltage', 'Voltage exceeds cell maximum threshold'),
35     ('Under_Voltage', 'Voltage below cell minimum threshold'),
36     ('Over_Temperature', 'Temperature exceeds cell maximum
37         threshold'),
38     ('Under_Temperature', 'Temperature below safe operating
39         minimum'),
40     ('Over_Current', 'Current draw exceeds safe continuous
41         rating'),
42     ('SOH_Critical', 'State of Health dropped to critically low
43         level'),
44     ('SOC_Anomaly', 'State of Charge reading is inconsistent');
45
46 CREATE TABLE FaultSeverityLookup (
47     code VARCHAR(10) PRIMARY KEY, priority INT NOT NULL,
48     description TEXT
49 );
50 INSERT INTO FaultSeverityLookup VALUES
51     ('Warning', 1, 'Deviation <= 5% -- monitor closely'),
52     ('Critical', 2, 'Deviation 5-15% -- reduce load or controlled
53         shutdown'),
54     ('Emergency', 3, 'Deviation > 15% -- immediate shutdown
55         required');

```

Listing B.1: Section 1 — Lookup / Reference Tables

```

1 CREATE TABLE BatteryPack (
2     pack_id SERIAL PRIMARY KEY,
3     pack_name VARCHAR(100) NOT NULL,
4     application_type VARCHAR(20) NOT NULL REFERENCES
5         ApplicationType(code),
6     total_nominal_voltage NUMERIC(8,2),
7     total_capacity_kwh NUMERIC(10,3),
8     installation_date DATE,
9     location VARCHAR(200)
10 );
11
12 CREATE TABLE BatteryCell (
13     battery_id SERIAL PRIMARY KEY,
14     pack_id INT NOT NULL
15         REFERENCES BatteryPack(pack_id) ON DELETE CASCADE,
16     manufacturer VARCHAR(100),
17     manufacture_date DATE,
18     nominal_voltage NUMERIC(5,3) NOT NULL,
19     nominal_capacity_ah NUMERIC(8,3) NOT NULL,

```

```

19     max_voltage          NUMERIC(5,3) NOT NULL,
20     min_voltage          NUMERIC(5,3) NOT NULL,
21     max_temperature      NUMERIC(5,1) NOT NULL,
22     chemistry_type       VARCHAR(5)   NOT NULL REFERENCES
        ChemistryType(code),
23     installation_date    DATE,
24     status               VARCHAR(10)  NOT NULL DEFAULT 'Active'
25                           REFERENCES CellStatus(code),
26     CONSTRAINT chk_voltage_range
27         CHECK (min_voltage < nominal_voltage AND nominal_voltage <
        max_voltage)
28 );
29
30 -- EER Subclasses
31 CREATE TABLE NMC_Cell (
32     battery_id           INT PRIMARY KEY
33         REFERENCES BatteryCell(battery_id) ON DELETE CASCADE,
34     nickel_ratio NUMERIC(4,3), manganese_ratio NUMERIC(4,3),
35     cobalt_ratio NUMERIC(4,3), energy_density_wh_kg NUMERIC(6,2),
36     max_charge_rate_c NUMERIC(4,2),
37     CONSTRAINT chk_nmc_ratios CHECK (
38         ROUND(COALESCE(nickel_ratio,0)+COALESCE(manganese_ratio,0)
39             +COALESCE(cobalt_ratio,0),2) <= 1.01)
40 );
41 CREATE TABLE LFP_Cell (
42     battery_id INT PRIMARY KEY
43         REFERENCES BatteryCell(battery_id) ON DELETE CASCADE,
44     iron_phosphate_purity NUMERIC(5,2),
45     thermal_stability_rating VARCHAR(2)
46         CHECK (thermal_stability_rating IN ('A','B','C','D')),
47     recommended_dod NUMERIC(5,2)
48 );
49 CREATE TABLE LTO_Cell (
50     battery_id INT PRIMARY KEY
51         REFERENCES BatteryCell(battery_id) ON DELETE CASCADE,
52     titanate_grade VARCHAR(20), rated_cycle_life INT,
53     fast_charge_capability_c NUMERIC(4,2)
54 );
55
56 CREATE TABLE Sensor (
57     sensor_id          SERIAL          PRIMARY KEY,
58     battery_id         INT             NOT NULL
59         REFERENCES BatteryCell(battery_id) ON DELETE CASCADE,
60     sensor_type        VARCHAR(15) NOT NULL
61         CHECK (sensor_type IN ('Voltage','Current','Temperature')),
62     calibration_date   DATE,
63     accuracy_rating    NUMERIC(5,3)

```

```

64 );
65
66 CREATE TABLE SensorReading (
67     reading_id          BIGSERIAL          PRIMARY KEY,
68     sensor_id           INT                 NOT NULL
69         REFERENCES Sensor(sensor_id) ON DELETE CASCADE,
70     reading_timestamp    TIMESTAMPTZ       NOT NULL DEFAULT NOW(),
71     value                NUMERIC(10,4)     NOT NULL,
72     unit                 VARCHAR(20)       NOT NULL
73 );
74 CREATE INDEX idx_reading_ts          ON
75     SensorReading(reading_timestamp DESC);
76 CREATE INDEX idx_reading_sensor_ts  ON SensorReading(sensor_id,
77     reading_timestamp DESC);
78 CREATE INDEX idx_reading_brin        ON SensorReading USING
79     BRIN(reading_timestamp);
80
81 CREATE TABLE ChargingProfile (
82     profile_id SERIAL PRIMARY KEY, profile_name VARCHAR(100) NOT
83     NULL,
84     chemistry_type VARCHAR(5) NOT NULL REFERENCES
85     ChemistryType(code),
86     min_soc_range NUMERIC(5,2), max_soc_range NUMERIC(5,2),
87     min_soh_range NUMERIC(5,2), max_soh_range NUMERIC(5,2),
88     recommended_charge_rate_c NUMERIC(4,2) NOT NULL,
89     recommended_max_voltage NUMERIC(5,3),
90     temp_range_min NUMERIC(5,1), temp_range_max NUMERIC(5,1),
91     description TEXT,
92     CONSTRAINT chk_profile_soc CHECK (min_soc_range <=
93     max_soc_range),
94     CONSTRAINT chk_profile_soh CHECK (min_soh_range <=
95     max_soh_range)
96 );
97
98 CREATE TABLE ChargeCycle (
99     cycle_id SERIAL PRIMARY KEY,
100    battery_id INT NOT NULL REFERENCES BatteryCell(battery_id) ON
101        DELETE CASCADE,
102    cycle_type VARCHAR(10) NOT NULL CHECK (cycle_type IN
103        ('Charge', 'Discharge')),
104    start_time TIMESTAMPTZ NOT NULL, end_time TIMESTAMPTZ,
105    start_soc NUMERIC(5,2), end_soc NUMERIC(5,2),
106    start_voltage NUMERIC(5,3), end_voltage NUMERIC(5,3),
107    avg_current NUMERIC(8,3), avg_temperature NUMERIC(5,1),
108    energy_throughput_wh NUMERIC(10,3),
109    charging_profile_id INT REFERENCES ChargingProfile(profile_id)
110        ON DELETE SET NULL

```



```

101 );
102
103 CREATE TABLE BatteryHealthLog (
104     log_id SERIAL PRIMARY KEY,
105     battery_id INT NOT NULL REFERENCES BatteryCell(battery_id) ON
        DELETE CASCADE,
106     log_timestamp TIMESTAMPTZ NOT NULL DEFAULT NOW(),
107     soc_percentage NUMERIC(5,2),
108     soh_percentage NUMERIC(5,2) NOT NULL CHECK (soh_percentage
        BETWEEN 0 AND 100),
109     internal_resistance_mohm NUMERIC(8,3),
110     capacity_remaining_ah NUMERIC(8,3),
111     cycle_count INT
112 );
113
114 CREATE TABLE FaultLog (
115     fault_id SERIAL PRIMARY KEY,
116     battery_id INT NOT NULL REFERENCES BatteryCell(battery_id) ON
        DELETE CASCADE,
117     fault_type VARCHAR(20) NOT NULL REFERENCES
        FaultTypeLookup(code),
118     severity VARCHAR(10) NOT NULL REFERENCES
        FaultSeverityLookup(code),
119     detected_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
120     resolved_at TIMESTAMPTZ, description TEXT,
121     trigger_reading_id BIGINT
122     REFERENCES SensorReading(reading_id) ON DELETE SET NULL
123 );
124 CREATE INDEX idx_fault_unresolved ON FaultLog(battery_id)
125     WHERE resolved_at IS NULL;
126
127 CREATE TABLE SensorReadingSummary (
128     summary_id BIGSERIAL PRIMARY KEY,
129     sensor_id INT NOT NULL REFERENCES Sensor(sensor_id) ON DELETE
        CASCADE,
130     summary_date DATE NOT NULL, summary_hour SMALLINT
        CHECK (summary_hour BETWEEN 0 AND 23),
131     min_value NUMERIC(10,4), max_value NUMERIC(10,4),
132     avg_value NUMERIC(10,4), stddev_value NUMERIC(10,4),
133     reading_count INT,
134     CONSTRAINT uq_summary_unique UNIQUE (sensor_id, summary_date,
        summary_hour)
135 );
136 );

```

Listing B.2: Section 2 — Core Tables and Indexes

Appendix C

Remaining Analytics Queries (Q2, Q4, Q5, Q6, Q7)

```
1 WITH soh_changes AS (  
2     SELECT bhl.battery_id, bc.chemistry_type, bhl.log_timestamp,  
3           bhl.soh_percentage, bhl.cycle_count,  
4           LAG(bhl.soh_percentage) OVER  
5             (PARTITION BY bhl.battery_id ORDER BY  
6               bhl.log_timestamp) AS prev_soh,  
7           LAG(bhl.log_timestamp) OVER  
8             (PARTITION BY bhl.battery_id ORDER BY  
9               bhl.log_timestamp) AS prev_ts  
10    FROM BatteryHealthLog bhl  
11    JOIN BatteryCell bc ON bhl.battery_id = bc.battery_id  
12 ),  
13 flagged AS (  
14     SELECT *, (prev_soh - soh_percentage) AS soh_drop,  
15           EXTRACT(DAY FROM (log_timestamp - prev_ts)) AS  
16             days_elapsed,  
17           ROUND((prev_soh - soh_percentage) /  
18             NULLIF(EXTRACT(DAY FROM (log_timestamp -  
19               prev_ts)),0),4) AS drop_per_day  
20     FROM soh_changes WHERE prev_soh IS NOT NULL  
21 )  
22 SELECT battery_id, chemistry_type, prev_ts AS period_start,  
23        log_timestamp AS period_end, prev_soh, soh_percentage,  
24        soh_drop, drop_per_day, cycle_count, days_elapsed  
25 FROM flagged WHERE soh_drop > 3.00  
26 ORDER BY soh_drop DESC;
```

Listing C.1: Q2: Abnormal SoH drop detection (drop > 3%)

```
1 WITH classified_cycles AS (  
2     SELECT cc.battery_id, bc.chemistry_type,
```

```

3         cc.avg_current, cc.avg_temperature,
4         cc.energy_throughput_wh,
5         CASE WHEN cp.recommended_charge_rate_c >= 1.00 THEN
6             'Fast'
7         ELSE 'Standard' END AS charge_category
8 FROM ChargeCycle cc
9 JOIN BatteryCell bc ON cc.battery_id = bc.battery_id
10 LEFT JOIN ChargingProfile cp ON cc.charging_profile_id =
11     cp.profile_id
12 WHERE cc.cycle_type = 'Charge'
13 )
14 SELECT chemistry_type, charge_category,
15        COUNT(*) AS cycle_count,
16        ROUND(AVG(avg_temperature),2) AS avg_temp_c,
17        ROUND(MAX(avg_temperature),2) AS max_temp_c,
18        ROUND(AVG(avg_current),2) AS avg_current_a,
19        ROUND(AVG(energy_throughput_wh),2) AS avg_energy_wh,
20        ROUND(STDDEV(avg_temperature),3) AS temp_stddev
21 FROM classified_cycles
22 GROUP BY chemistry_type, charge_category
23 ORDER BY chemistry_type, charge_category;

```

Listing C.2: Q4: Fast-charging vs standard charging impact

```

1 WITH fault_stats AS (
2     SELECT bc.chemistry_type, fl.fault_type, fl.severity,
3           COUNT(*) AS fault_count,
4           SUM(CASE WHEN fl.resolved_at IS NOT NULL THEN 1 ELSE 0
5               END) AS resolved,
6           ROUND(AVG(EXTRACT(EPOCH FROM (fl.resolved_at -
7               fl.detected_at)))/60.0)
8               FILTER (WHERE fl.resolved_at IS NOT NULL),2) AS
9               avg_res_min
10 FROM FaultLog fl
11 JOIN BatteryCell bc ON fl.battery_id = bc.battery_id
12 GROUP BY bc.chemistry_type, fl.fault_type, fl.severity
13 )
14 SELECT chemistry_type, fault_type, severity, fault_count,
15        resolved, avg_res_min,
16        ROUND(100.0*fault_count/SUM(fault_count) OVER (PARTITION BY
17            chemistry_type),2)
18            AS pct_of_chemistry_faults,
19        SUM(fault_count) OVER (PARTITION BY chemistry_type
20            ORDER BY fault_count DESC ROWS UNBOUNDED PRECEDING) AS
21            running_total
22 FROM fault_stats
23 ORDER BY chemistry_type, fault_count DESC;

```

Listing C.3: Q5: Fault frequency by chemistry and severity

```
1 WITH daily_avg AS (  
2     SELECT s.battery_id, bc.chemistry_type, s.sensor_type,  
3           DATE(sr.reading_timestamp AT TIME ZONE 'Asia/Karachi')  
4           AS reading_date,  
5           ROUND(AVG(sr.value),4) AS daily_avg_value  
6     FROM SensorReading sr  
7     JOIN Sensor s ON sr.sensor_id = s.sensor_id  
8     JOIN BatteryCell bc ON s.battery_id = bc.battery_id  
9     GROUP BY s.battery_id, bc.chemistry_type, s.sensor_type,  
10            DATE(sr.reading_timestamp AT TIME ZONE 'Asia/Karachi')  
11 )  
12 SELECT battery_id, chemistry_type, sensor_type, reading_date,  
13        daily_avg_value,  
14        ROUND(AVG(daily_avg_value) OVER (  
15            PARTITION BY battery_id, sensor_type ORDER BY reading_date  
16            ROWS BETWEEN 6 PRECEDING AND CURRENT ROW  
17        ),4) AS rolling_7day_avg  
18 FROM daily_avg  
19 ORDER BY battery_id, sensor_type, reading_date;
```

Listing C.4: Q7: Rolling 7-day average sensor reading

Syed Ammar Zulfiqar

Smart Battery Management System

 Quick Submit Quick Submit National University of Computer and Emerging Sciences, Islamabad

Document Details

Submission ID

trn:oid:::1:3555583252

Submission Date

May 1, 2026, 9:11 AM GMT+5

Download Date

May 1, 2026, 9:13 AM GMT+5

File Name

FDB_CEP_REPORT_FDB.pdf

File Size

2.44 MB

36 Pages

2,850 Words

21,400 Characters

0% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Detection Groups

 **0 AI-generated only 0%**
Likely AI-generated text from a large-language model.

 **0 AI-generated text that was AI-paraphrased 0%**
Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



Smart Battery Management System

ORIGINALITY REPORT

0%

SIMILARITY INDEX

0%

INTERNET SOURCES

0%

PUBLICATIONS

0%

STUDENT PAPERS

PRIMARY SOURCES

Exclude quotes On

Exclude bibliography On

Exclude matches < 3%